UNIVERSIDADE PAULISTA – UNIP
CIÊNCIA DA COMPUTAÇÃO – 7º/8º SEMESTRE
ATIVIDADES PRÁTICAS SUPERVISIONADAS (APS)

Bruno Nascimento de Paula Silva – RA G793030
[NOME COMPLETO DO ALUNO 2] – RA [RA]
[NOME COMPLETO DO ALUNO 3] – RA [RA]

VIAAGORA

Aplicação de sistema distribuído para dispositivo móvel com o tempo de espera das travessias litorâneas da Baixada Santista

Tema: Desenvolvimento de uma aplicação de sistema distribuído para dispositivo móvel

Disciplina vinculada: Desenvolvimento de Sistemas Distribuídos (DSD)

[CIDADE]
2026

SUMÁRIO

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO 4
2 INTRODUÇÃO 6
2.1 Escopo 7
2.2 Interdisciplinaridade 7
2.3 Organização do trabalho 8
3 FUNDAMENTOS DAS TECNOLOGIAS PARA DISPOSITIVOS MÓVEIS ESCOLHIDAS 9
3.1 Plataforma Android 9
3.2 Linguagem Kotlin 9
3.3 Comunicação entre o aplicativo e o servidor 10
3.4 Mapas e localização 10
3.5 Conceitos de sistemas distribuídos 10
3.6 Middleware orientado a mensagens: AMQP e RabbitMQ 11
3.7 Persistência: PostgreSQL e transações 11
3.8 Servidor: Node.js, Fastify e Zod 12
3.9 Coleta de dados da web (web scraping) 12
3.10 Contêineres e Docker Compose 12
4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO 14
4.1 Requisitos funcionais 14
4.2 Requisitos não funcionais 14
4.3 Ferramentas e tecnologias 15
4.4 Etapas de desenvolvimento 16
4.5 Estratégia de testes 16
4.6 Riscos e mitigação 17
4.7 Perfis de usuário e casos de uso 17
4.8 Decisões de projeto 18
4.9 Segurança e privacidade 19
4.10 Organização do código e controle de versões 19
4.11 Ambiente de execução 19
5 PROJETO DO PROGRAMA 21
5.1 Visão geral da arquitetura 21
5.2 Módulos do servidor 21
5.3 Modelo de dados 22
5.4 Interface de programação (API) 22
5.5 Módulos do aplicativo Android 22
5.6 Tratamento de erros 23

5.7 Fluxos principais 24
6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA 25
6.1 Varredura do site oficial 26
6.2 Coleta com tolerância a falhas 27
6.3 Gravação das leituras com outbox transacional 28
6.4 Publicação dos eventos e entrega às réplicas 28
6.5 Comunicação no aplicativo Android 29
7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO 31
7.1 Preparação (antes da apresentação) 31
7.2 Roteiro 31
7.3 Demonstração das falhas 33
8 CONSIDERAÇÕES FINAIS 34
REFERÊNCIAS 35

1 OBJETIVO E MOTIVAÇÃO DO TRABALHO

O objetivo deste trabalho é desenvolver o **ViaAgora**, uma aplicação de sistema distribuído para dispositivo móvel que informa, em tempo real, o tempo de espera das travessias litorâneas da Baixada Santista: Santos ⇄ Guarujá, Santos ⇄ Vicente de Carvalho e Bertioga ⇄ Guarujá. A aplicação reúne duas fontes de informação. A primeira é o painel público das Travessias Litorâneas, mantido pela Secretaria de Meio Ambiente, Infraestrutura e Logística do Estado de São Paulo (SEMIL-SP). A segunda são os relatos dos próprios usuários, que informam quanto tempo estão esperando na fila e se há embarque lento, travessia parada, neblina, mar agitado ou acidente no acesso.

São objetivos específicos do projeto:

- construir um coletor que varre o site oficial, descobre o serviço que alimenta o painel e transforma esses dados em uma API própria, com histórico;
- distribuir o processamento entre processos independentes (coletor, réplicas da API, worker de eventos, broker de mensagens e banco de dados), tolerando a falha de qualquer um deles sem perda de dados;
- entregar as alterações a todos os aparelhos conectados em tempo real, por WebSocket, independentemente da réplica da API em que cada um esteja ligado;
- oferecer funções extras de apoio à decisão: relatório do melhor horário para atravessar, avisos de espera e mapa dos terminais;
- desenvolver um aplicativo Android nativo, testado em emulador, com modo de demonstração que funciona sem servidor.

A motivação vem do cotidiano da região. As balsas são um serviço de transporte público usado diariamente por trabalhadores, estudantes e turistas, e por muitos é o único caminho prático entre as cidades. Nos horários de pico, finais de semana e feriados, as filas de veículos ocupam as avenidas de acesso, aumentam o congestionamento urbano e mantêm motores ligados por longos períodos, com emissão de poluentes. A operação também depende de condições ambientais: a maré baixa pode impedir o transporte de veículos pesados, e a neblina e o mar agitado reduzem a frequência das viagens.

O painel oficial informa a espera atual, mas não guarda histórico, não avisa quando a fila diminui e não considera a percepção de quem já está na fila. Um sistema que combina o dado oficial com os relatos da comunidade, guarda o histórico e avisa o usuário no momento certo ajuda a escolher o melhor horário. Isso reduz o tempo parado na fila e, por consequência, o impacto no trânsito e no ar da cidade. O tema se enquadra no item da APS que trata de informações ambientais urbanas, que cita explicitamente o "trânsito (inclusive transportes públicos)".

Do ponto de vista acadêmico, o projeto foi escolhido porque exige, de forma natural, os principais conceitos da disciplina de Desenvolvimento de Sistemas Distribuídos: comunicação assíncrona por mensagens, replicação, consistência entre banco de dados e mensagens, idempotência, tolerância a falhas e integração com uma fonte externa que não está sob controle do grupo.

2 INTRODUÇÃO

A Baixada Santista é uma região metropolitana formada por nove municípios do litoral paulista, recortada por canais, estuários e ilhas. Santos e Guarujá ficam em lados opostos do canal do porto. Guarujá ocupa a Ilha de Santo Amaro e Bertioga está separada de Guarujá pelo Canal de Bertioga. Por essa geografia, as travessias por balsa e por embarcações de passageiros fazem parte da mobilidade urbana da região há mais de um século. O sistema de Travessias Litorâneas do Estado de São Paulo é administrado pela Coordenadoria de Travessias da SEMIL e informa, em seu site, o tempo de espera em cada ponto, o número de embarcações em operação, as condições do tempo e avisos aos usuários.

Os sistemas de informação ao usuário de transporte público evoluíram de quadros de horários estáticos para serviços em tempo real, que combinam dados de operação, sensores e a contribuição dos próprios passageiros. Essa última fonte, conhecida como **crowdsourcing**, tem a vantagem de capturar o que acontece na rua, como o tamanho real da fila no acesso ao terminal, mas exige tratamento: os relatos podem ser imprecisos, repetidos ou chegar fora de ordem. Combinar dados oficiais e colaborativos, mantendo a origem de cada um visível ao usuário, é um problema típico de sistemas distribuídos.

Um sistema distribuído é, segundo Tanenbaum e Van Steen (2007), um conjunto de computadores independentes que se apresenta ao usuário como um sistema único e coerente. No ViaAgora, essa definição se concretiza em seis processos que rodam de forma independente em contêineres: um coletor que lê o painel oficial, duas réplicas da API que atendem os celulares, um worker que publica eventos, o broker de mensagens RabbitMQ e o banco de dados PostgreSQL. Do ponto de vista do usuário, há apenas um aplicativo, que mostra as travessias e se atualiza sozinho.

O problema central do trabalho é manter esse conjunto coerente diante de falhas. A fonte oficial pode sair do ar ou mudar de formato, o broker pode ficar indisponível, uma réplica da API pode reiniciar e o celular pode perder a conexão. O projeto adota técnicas consolidadas para cada caso: o banco de dados é a fonte de verdade, os eventos são gravados na mesma transação dos dados (padrão outbox transacional), cada réplica recebe todos os eventos por meio de uma exchange do

tipo fanout, as requisições de escrita são idempotentes e os clientes sempre buscam o estado atual depois de uma notificação ou de uma reconexão.

2.1 Escopo

O escopo abrange as três travessias da Baixada Santista acompanhadas pelo painel oficial: Santos ⇄ Guarujá e Bertioga ⇄ Guarujá, para veículos e pedestres, e Santos ⇄ Vicente de Carvalho, para pedestres e ciclistas. O aplicativo permite consultar a espera de cada lado, ver o número de embarcações, o tempo e os avisos, publicar e encerrar relatos, consultar o histórico por hora do dia e criar avisos de espera. Ficam fora do escopo a compra de bilhetes, a integração com o agendamento oficial (hora marcada), a moderação de relatos por uma autoridade e a hospedagem pública do servidor.

2.2 Interdisciplinaridade

O trabalho integra conteúdos de diversas disciplinas do curso. De **Desenvolvimento de Sistemas Distribuídos** vêm a arquitetura em processos independentes, a comunicação por mensagens, a replicação e as estratégias de consistência e tolerância a falhas. De **Redes de Computadores** vêm os protocolos HTTP, WebSocket e AMQP e o endereçamento entre o celular, o computador e os contêineres. De **Banco de Dados** vêm a modelagem relacional, as transações ACID, o controle de concorrência com bloqueio de linhas e as consultas agregadas do histórico.

De **Engenharia de Software** vêm o levantamento de requisitos, a arquitetura em módulos, os testes automatizados e o controle de versões. De **Programação para Dispositivos Móveis** vêm o ciclo de vida de uma Activity, as permissões de localização e notificação, o acesso à rede fora da thread principal e o desenho de interfaces. De **Estatística** vêm a mediana usada na estimativa da comunidade, que é robusta a valores extremos, e as médias por hora que formam o relatório de melhor horário. Por fim, o tema dialoga com **Meio Ambiente e Mobilidade Urbana**: filas menores significam menos congestionamento e menos emissões, e as condições do mar e do tempo são exibidas porque afetam diretamente a operação.

2.3 Organização do trabalho

A seção 3 apresenta os fundamentos das tecnologias móveis e distribuídas escolhidas. A seção 4 descreve o plano de desenvolvimento, com requisitos, ferramentas, etapas, testes e riscos. A seção 5 detalha o projeto do programa, seus módulos e contratos. A seção 6 traz o relatório com as linhas de código mais importantes. A seção 7 contém o roteiro da apresentação do programa em funcionamento, e a seção 8 apresenta as considerações finais, incluindo o efeito deste trabalho na formação do grupo.

3 FUNDAMENTOS DAS TECNOLOGIAS PARA DISPOSITIVOS MÓVEIS ESCOLHIDAS

3.1 Plataforma Android

O Android é um sistema operacional de código aberto baseado no núcleo Linux e mantido pelo Google, presente na maior parte dos smartphones em uso no Brasil. Os aplicativos são distribuídos em pacotes APK e executados pelo Android Runtime (ART), que compila o código intermediário para instruções nativas do aparelho. Cada aplicativo roda em um processo isolado, com um usuário próprio do sistema, e só acessa recursos sensíveis (localização, notificações, internet) se declarar as permissões correspondentes no arquivo de manifesto. Permissões consideradas perigosas, como a localização e, a partir do Android 13, as notificações, também precisam ser concedidas pelo usuário durante o uso.

A unidade básica de interface é a **Activity**, que passa por um ciclo de vida controlado pelo sistema: criação, início, retomada, pausa, parada e destruição. Um aplicativo bem construído libera recursos e pausa tarefas periódicas quando sai de primeiro plano e as retoma ao voltar. O sistema também proíbe operações de rede na thread principal, responsável pela interface, para que o aplicativo não congele. Por isso as chamadas ao servidor precisam ser feitas em threads de trabalho e os resultados devolvidos à thread principal. O ViaAgora declara como versão mínima o Android 8 (API 26) e como alvo o Android 15 (API 35).

3.2 Linguagem Kotlin

O Kotlin é uma linguagem moderna, estaticamente tipada, que roda na máquina virtual Java e é a linguagem recomendada pelo Google para o desenvolvimento Android desde 2019. Entre seus recursos estão as classes de dados (data classes), que geram automaticamente igualdade, cópia e representação textual; a segurança contra nulos no próprio sistema de tipos, que reduz falhas por referência nula; as funções de ordem superior e expressões lambda, usadas para tratar respostas assíncronas; e a interoperabilidade total com bibliotecas Java. No projeto, o Kotlin foi usado na versão 2.0.21, com compilação pelo Android Gradle Plugin 8.7.3 e Gradle 8.9.

3.3 Comunicação entre o aplicativo e o servidor

A comunicação de consulta e escrita usa **HTTP** com mensagens em **JSON**, seguindo o estilo arquitetural REST descrito por Fielding (2000): cada recurso (travessias, relatos, participantes) tem um endereço próprio e é manipulado pelos métodos GET, POST e PATCH. O HTTP é um protocolo de requisição e resposta: o cliente pergunta e o servidor responde. Ele não permite, sozinho, que o servidor avise o cliente de uma mudança.

Para as notificações em tempo real foi usado o protocolo **WebSocket** (FETTE; MELNIKOV, 2011). Ele começa como uma requisição HTTP e, depois de um aceite (handshake), mantém um canal TCP bidirecional aberto entre o celular e o servidor. Por esse canal o servidor envia, a qualquer momento, mensagens curtas avisando que algo mudou. O aplicativo usa a biblioteca **OkHttp** 4.12, que implementa HTTP, WebSocket, tempos limite e envio periódico de ping para detectar conexões interrompidas.

3.4 Mapas e localização

O mapa usa o **MapLibre Native**, biblioteca de código aberto derivada do Mapbox GL, que desenha mapas no aparelho com aceleração gráfica. O fundo cartográfico vem do **OpenStreetMap**, uma base colaborativa e livre que exige atribuição. Os terminais e os relatos são desenhados como uma camada de círculos sobre uma fonte GeoJSON, formato padrão para dados geográficos. A posição do usuário é obtida pelo LocationManager do Android, aceitando apenas leituras recentes, para que um relato não seja preenchido com uma posição antiga.

3.5 Conceitos de sistemas distribuídos

Coulouris et al. (2013) destacam como desafios de um sistema distribuído a heterogeneidade, a abertura, a segurança, a escalabilidade, o tratamento de falhas, a concorrência e a transparência. A **escalabilidade** é obtida quando é possível acrescentar instâncias de um componente para atender mais usuários. No ViaAgora isso é demonstrado com duas réplicas da API, que não guardam estado próprio e podem ser multiplicadas. A **transparência** faz o usuário ignorar em qual réplica está conectado, porque todas mostram o mesmo estado.

O **tratamento de falhas** parte do princípio de que componentes falham de forma independente: a queda do broker não deve impedir a gravação de um relato, e a queda da fonte oficial não deve derrubar o aplicativo. Para isso são usadas técnicas como a persistência antes da notificação, as novas tentativas com intervalo crescente (backoff exponencial) e a reconciliação de estado após uma reconexão. Kleppmann (2017) chama atenção para o fato de que, em redes reais, mensagens podem ser perdidas, atrasadas ou duplicadas. Por isso o sistema deve tolerar a reentrega (entrega "pelo menos uma vez") e tornar as operações **idempotentes**, isto é, seguras para repetir.

3.6 Middleware orientado a mensagens: AMQP e RabbitMQ

Um middleware orientado a mensagens desacopla quem produz um evento de quem o consome. O produtor não precisa saber quantos consumidores existem nem se estão disponíveis no momento. O **RabbitMQ** é um broker de mensagens de código aberto que implementa o protocolo **AMQP 0-9-1**. Nesse modelo, os produtores publicam em uma **exchange**, que roteia as mensagens para **filas** conforme regras de ligação (bindings). Os consumidores leem as filas e confirmam o processamento.

A exchange do tipo **fanout** copia cada mensagem para todas as filas ligadas a ela, implementando o padrão publish/subscribe (HOHPE; WOOLF, 2003). Esse é o comportamento necessário quando várias réplicas da API precisam avisar seus próprios clientes. Se todas consumissem uma única fila, as mensagens seriam divididas entre elas, e parte dos celulares não seria avisada. O RabbitMQ também oferece **confirmações do publicador** (publisher confirms), que garantem ao produtor que a mensagem foi aceita pelo broker antes de ela ser considerada publicada.

3.7 Persistência: PostgreSQL e transações

O **PostgreSQL** é um sistema gerenciador de banco de dados relacional de código aberto, com suporte completo a transações **ACID**: atomicidade, consistência, isolamento e durabilidade. Uma transação agrupa várias operações que são confirmadas juntas ou desfeitas juntas. O projeto também usa o bloqueio de linhas com SELECT ... FOR UPDATE SKIP LOCKED, que permite a vários processos consumirem uma tabela de trabalho sem pegar o mesmo item, e funções de agregação como avg e percentile_cont, usadas no histórico e na mediana da comunidade.

O padrão **outbox transacional** (RICHARDSON, 2018) resolve um problema clássico: como garantir que um dado salvo no banco gere a mensagem correspondente no broker, se são dois sistemas diferentes? A solução é gravar o evento em uma tabela do próprio banco, na mesma transação do dado. Um processo separado lê essa tabela e publica no broker. Se o broker estiver fora do ar, os eventos ficam pendentes e são publicados quando ele voltar, sem perda.

3.8 Servidor: Node.js, Fastify e Zod

O servidor foi escrito em JavaScript sobre o **Node.js** 22, um ambiente de execução orientado a eventos com entrada e saída não bloqueantes, adequado a serviços que passam a maior parte do tempo esperando a rede e o banco. O framework **Fastify** 5 organiza as rotas HTTP, com limite de taxa de requisições por meio do plugin @fastify/rate-limit. A biblioteca **Zod** 4 valida os dados de entrada e os dados vindos da fonte oficial, rejeitando valores fora do formato esperado. As bibliotecas pg, amqplib e ws fazem a ligação com o PostgreSQL, o RabbitMQ e o WebSocket.

3.9 Coleta de dados da web (web scraping)

A coleta automatizada de dados de páginas web, conhecida como web scraping, consiste em obter o conteúdo de uma página e extrair dele informações estruturadas. Em páginas modernas, muitos dados não estão no HTML: são carregados depois por scripts que consultam serviços em segundo plano. Nesses casos, o coletor precisa reproduzir o caminho do navegador, isto é, ler a página, encontrar o script responsável e descobrir o serviço consultado. Uma coleta responsável limita a frequência de acesso, identifica-se, usa um único ponto de coleta em vez de cada usuário acessar a fonte e atribui a origem dos dados.

3.10 Contêineres e Docker Compose

Um contêiner empacota um processo com suas dependências e o executa isolado do sistema hospedeiro, de forma mais leve que uma máquina virtual. O **Docker Compose** descreve, em um único arquivo, vários contêineres, suas redes, volumes e dependências, e permite subir todo o ambiente com um comando. No projeto, o Compose inicia o banco, o broker, as duas réplicas da API, o worker e o

coletor, com verificações de saúde (healthchecks) e reinício automático. Os dados ficam em volumes, preservados entre reinicializações.

4 PLANO DE DESENVOLVIMENTO DA APLICAÇÃO

Esta seção descreve os elementos e ferramentas usados no desenvolvimento: os requisitos que orientaram o trabalho, as tecnologias escolhidas e a justificativa de cada uma, as etapas de execução, a estratégia de testes, os riscos identificados e o ambiente de execução.

4.1 Requisitos funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer. Eles foram levantados a partir do problema descrito na seção 1 e do painel oficial das travessias.

Tabela 1 – Requisitos funcionais

Código	Requisito
RF01	Coletar periodicamente o painel oficial das travessias e guardar cada leitura (espera de cada lado, embarcações, tempo, período e aviso).
RF02	Exibir, para cada travessia, a espera atual de cada lado com classificação por cor (tranquila, moderada, longa ou sem operação).
RF03	Indicar a origem (painel oficial ou dado simulado) e a idade de cada leitura, sinalizando dados com mais de 10 minutos como atrasados.
RF04	Permitir publicar relatos com travessia, lado, tipo, espera informada (opcional), descrição (opcional) e ponto no mapa.
RF05	Calcular a estimativa da comunidade como a mediana das esperas informadas nos últimos 45 minutos, por lado.
RF06	Permitir que somente o autor encerre o próprio relato.
RF07	Notificar em tempo real todos os aparelhos conectados, em qualquer réplica, quando houver nova leitura ou relato.
RF08	Gerar o relatório de melhor horário: espera média por hora do dia nos últimos 7 dias e a melhor hora de cada lado entre 6h e 22h.
RF09	Permitir criar avisos de espera ("avise quando a espera ficar abaixo de N minutos"), disparados uma única vez.
RF10	Exibir mapa com os terminais coloridos pela espera e os relatos ativos.
RF11	Oferecer modo de demonstração local, com esperas simuladas, sem depender do servidor.

Fonte: elaborado pelos autores.

4.2 Requisitos não funcionais

Tabela 2 – Requisitos não funcionais

Código	Requisito
RNF01	Disponibilidade: a falha da fonte oficial, do broker ou de uma réplica não pode impedir a consulta nem a gravação de dados.
RNF02	Consistência: todo evento notificado corresponde a um dado já gravado; nenhum dado gravado deixa de gerar seu evento.

Código	Requisito
RNF03	Idempotência: o reenvio de um mesmo relato, inclusive simultâneo, cria um único registro.
RNF04	Segurança: identificação anônima por aparelho com token aleatório, guardado no banco apenas como hash SHA-256; limite de requisições por minuto.
RNF05	Respeito à fonte: um único coletor consulta o site oficial no máximo uma vez por minuto, com intervalo crescente em caso de falha.
RNF06	Usabilidade: interface em português, alvos de toque de pelo menos 48 dp e descrições para leitores de tela.
RNF07	Portabilidade: servidor executável com um único comando (Docker Compose); aplicativo compatível com Android 8 ou superior.

Fonte: elaborado pelos autores.

4.3 Ferramentas e tecnologias

A escolha das tecnologias priorizou ferramentas de código aberto, amplamente documentadas e adequadas a cada papel na arquitetura. A Tabela 3 resume os elementos utilizados.

Tabela 3 – Ferramentas e tecnologias utilizadas

Elemento	Versão	Uso no projeto
Kotlin	2.0.21	Linguagem do aplicativo Android.
Android SDK	API 26 a 35	Plataforma móvel; versão mínima Android 8, alvo Android 15.
Android Gradle Plugin / Gradle	8.7.3 / 8.9	Compilação, testes, lint e geração do APK.
OkHttp	4.12.0	Cliente HTTP e WebSocket do aplicativo.
MapLibre Native	11.8.0	Mapa com os terminais e relatos.
OpenStreetMap	—	Dados do fundo cartográfico.
Node.js	22	Ambiente de execução do servidor.
Fastify	5.6	Framework HTTP da API.
Zod	4.1	Validação de entradas e dos dados oficiais.
PostgreSQL	16	Banco de dados relacional, fonte de verdade.
RabbitMQ	4	Broker de mensagens (AMQP), exchange fanout.
ws / amqplib / pg	8.18 / 0.10 / 8.16	Bibliotecas de WebSocket, AMQP e PostgreSQL.
Docker e Docker Compose	28	Contêineres e orquestração local.
Android Emulator	API 33	Testes do aplicativo.
JUnit / node:test	4.13 / Node 22	Testes automatizados.
Git	—	Controle de versões.

Fonte: elaborado pelos autores.

4.4 Etapas de desenvolvimento

O desenvolvimento seguiu etapas incrementais. Cada uma terminou com testes automatizados executados contra o banco e o broker reais, e não contra simulações. A Tabela 4 apresenta as etapas na ordem em que foram realizadas.

Tabela 4 – Etapas de desenvolvimento

Etapa	Atividades	Resultado
1. Definição do problema	Leitura das regras da APS; escolha do tema de transporte público; análise do site das Travessias Litorâneas.	Proposta do projeto.
2. Investigação da fonte	Identificação do script do painel e do serviço de dados; registro de amostras reais para testes.	Formato dos dados conhecido.
3. Base distribuída	API, participantes, idempotência, outbox, worker, RabbitMQ e WebSocket com duas réplicas.	Infraestrutura testada.
4. Coletor	Varredura do site, validação, normalização, endereço de reserva, backoff e modo simulado.	Leituras gravadas com histórico.
5. Domínio das travessias	Relatos por travessia e lado, mediana da comunidade, dado atrasado e histórico por hora.	API completa.
6. Aplicativo Android	Telas de travessias, mapa e relatos; formulário; gráfico; avisos; modo demonstração.	APK assinado.
7. Verificação	Testes automatizados, lint e testes no emulador com os dados oficiais reais.	Relatório de verificação.
8. Documentação	Arquitetura, roteiro de demonstração e este trabalho escrito.	Entrega da APS.

Fonte: elaborado pelos autores.

4.5 Estratégia de testes

Foram adotados três níveis de verificação. O primeiro são os **testes de integração do servidor**: 20 testes executados com o node:test contra um PostgreSQL e um RabbitMQ reais, usando um banco separado com sufixo _test. Eles cobrem a validação dos relatos, a idempotência com seis envios simultâneos, a autorização do autor, a mediana da comunidade, o dado atrasado, o histórico, o coletor (com amostras reais do site gravadas como fixtures), a queda e a recuperação da fonte, a entrega de eventos a duas réplicas e a ressincronização após a perda da conexão com o broker. Essa última é simulada por um proxy TCP que é derrubado durante o teste.

O segundo nível são os **testes de unidade do Android**: 8 testes JUnit para a validação dos relatos, as faixas de espera, o disparo dos avisos, a simulação, o isolamento entre o modo local e o conectado e a normalização do endereço do servidor. O terceiro nível é a **verificação manual no emulador** Android 13: o aplicativo foi instalado, conectado ao servidor real e usado para publicar um relato, que foi recebido por um cliente ligado à outra réplica da API. Também foi executada a análise estática do Android Lint, sem erros.

4.6 Riscos e mitigação

Tabela 5 – Riscos identificados e mitigação

Risco	Impacto	Mitigação
O serviço oficial muda de endereço	Coleta interrompida	Varredura do site para redescobrir o endereço; endereço de reserva configurável.
A fonte oficial fica fora do ar ou bloqueia consultas	Dados desatualizados	Último dado preservado e sinalizado como atrasado; backoff até 10 min; modo simulado para a apresentação.
Queda do broker	Notificações atrasadas	Outbox transacional; dados gravados normalmente; publicação ao retornar.
Mensagens duplicadas ou perdidas	Estado incorreto no app	Eventos só disparam nova consulta; ressincronização em toda reconexão.
Reenvio de relatos pelo app	Registros duplicados	Chave de idempotência por envio, persistida no aparelho.
Relatos falsos ou exagerados	Estimativa distorcida	Mediana, janela de 45 minutos e origem sempre exibida.
Professor fora da rede local	Teste conectado inviável	Modo demonstração no APK; servidor publicável com HTTPS.

Fonte: elaborado pelos autores.

4.7 Perfis de usuário e casos de uso

Foram identificados dois perfis principais de usuário. O **usuário consultor** quer saber, antes de sair de casa, quanto tempo vai esperar e qual o melhor horário para atravessar. O **usuário colaborador** já está na fila e pode informar a situação real do acesso ao terminal. A mesma pessoa costuma assumir os dois papéis em momentos diferentes. Há ainda um terceiro ator, que não é humano: o **coletor**, que atua em nome do sistema obtendo os dados oficiais.

Tabela 6 – Casos de uso

Ator	Caso de uso	Descrição resumida
Usuário consultor	Consultar travessias	Ver a espera de cada lado, as embarcações, o tempo, os avisos e a estimativa da comunidade.

Ator	Caso de uso	Descrição resumida
Usuário consultor	Consultar melhor horário	Ver o gráfico da espera média por hora e a recomendação de cada lado.
Usuário consultor	Criar aviso de espera	Escolher a travessia, o lado e o limite em minutos; receber uma notificação quando a espera ficar dentro do limite.
Usuário consultor	Consultar mapa	Ver os terminais coloridos pela espera e os relatos ativos.
Usuário colaborador	Publicar relato	Informar travessia, lado, tipo, espera e detalhes a partir do terminal ou de um ponto do mapa.
Usuário colaborador	Encerrar relato	Marcar o próprio relato como encerrado quando a situação normalizar.
Coletor	Coletar painel oficial	Ler o painel a cada minuto, gravar a leitura e avisar os aparelhos quando houver mudança.

Fonte: elaborado pelos autores.

4.8 Decisões de projeto

Algumas decisões tiveram impacto direto na arquitetura e merecem justificativa. A primeira foi desenvolver um **aplicativo Android nativo** em vez de uma página web. O tema da APS pede uma aplicação para dispositivo móvel, e o aplicativo nativo dá acesso direto à localização, às notificações e ao mapa com aceleração gráfica, além de poder ser entregue ao professor como um APK.

A segunda foi **centralizar a coleta em um único processo** no servidor. Se cada celular consultasse o site oficial, o número de acessos cresceria com o número de usuários, e qualquer mudança no site exigiria atualizar todos os aparelhos. Com o coletor central, a fonte é consultada no máximo uma vez por minuto, os dados ganham histórico e a lógica de leitura pode ser corrigida em um único lugar.

A terceira foi usar um **broker de mensagens** em vez de fazer cada réplica consultar o banco periodicamente em busca de novidades. A consulta periódica (polling) gera carga constante mesmo sem mudanças e atrasa as notificações. Com o RabbitMQ, as réplicas são avisadas assim que um evento é publicado, e novas réplicas podem ser acrescentadas sem alterar as existentes. A quarta foi tratar o **PostgreSQL como fonte de verdade** e os eventos apenas como sinais para buscar o estado atual. Assim, uma mensagem perdida ou repetida nunca deixa o aplicativo com dados errados: no pior caso, a atualização chega na próxima consulta.

4.9 Segurança e privacidade

O sistema não coleta dados pessoais. A participação é anônima e criada por instalação: o servidor gera um token aleatório de 256 bits, entrega-o ao aparelho uma única vez e guarda apenas o seu hash SHA-256. Assim, mesmo quem tiver acesso ao banco não consegue se passar por um participante. O token é exigido apenas para publicar e encerrar relatos, e as respostas públicas nunca incluem tokens ou hashes. A localização do aparelho é usada só para posicionar o mapa e sugerir o ponto de um relato, e o usuário pode negar a permissão e escolher o ponto manualmente. Essas escolhas seguem o princípio da minimização de dados da Lei Geral de Proteção de Dados (Lei nº 13.709/2018).

Do lado do servidor, as entradas são validadas com esquemas estritos que rejeitam campos desconhecidos, o tamanho das requisições é limitado a 8 KB, as mensagens WebSocket dos clientes são limitadas a 1 KB e cada endereço pode fazer no máximo 300 requisições por minuto (15 por minuto para criar participações). O banco e o broker ficam acessíveis apenas ao próprio computador.

4.10 Organização do código e controle de versões

O código fica em um único repositório Git, organizado por componente: a pasta android contém o aplicativo; a pasta server contém a API, o coletor, o worker, os testes e a migração do banco; o arquivo compose.yaml descreve o ambiente completo; a pasta docs reúne a documentação, incluindo este trabalho; a pasta scripts traz os scripts de compilação do APK e de início do servidor; e a pasta releases recebe o APK gerado. Arquivos gerados, dependências baixadas, chaves de assinatura e o arquivo .env com senhas locais não são versionados.

4.11 Ambiente de execução

O servidor é executado com Docker Compose em um computador com Docker Desktop. As réplicas da API ficam nas portas 4310 e 4311. O PostgreSQL e o RabbitMQ são expostos apenas ao próprio computador, de modo que o celular acessa somente a API. O aplicativo pode ser instalado em um celular Android na mesma rede Wi-Fi, configurado com o endereço IP do computador, ou executado no emulador, que acessa o computador pelo endereço especial 10.0.2.2. Para avaliação fora da rede

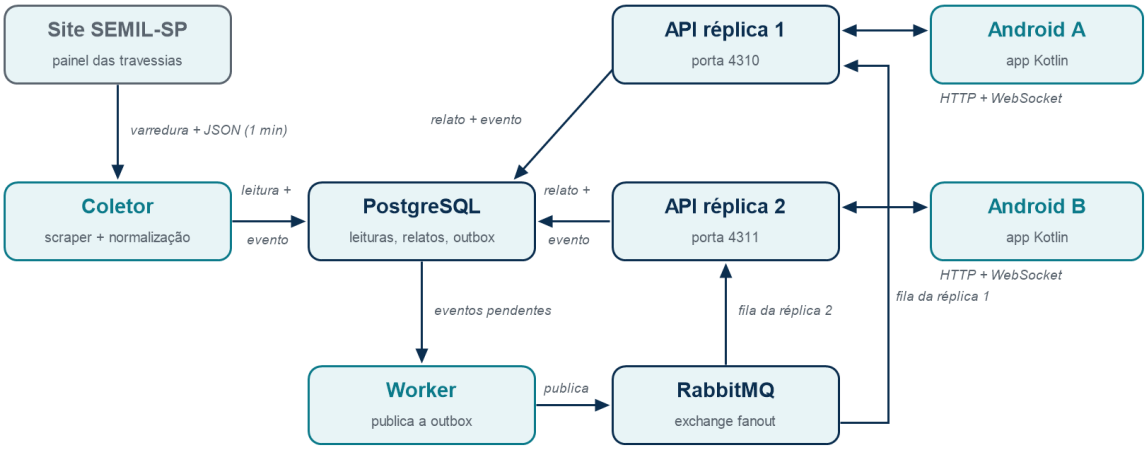
local, o servidor precisaria ser publicado em um endereço acessível pela internet, com HTTPS.

5 PROJETO DO PROGRAMA

5.1 Visão geral da arquitetura

A Figura 1 apresenta a arquitetura do ViaAgora. O sistema é formado por seis processos independentes, além dos aplicativos. O PostgreSQL é a fonte de verdade: tanto o coletor quanto as APIs gravam primeiro no banco, junto com o evento correspondente na tabela de outbox. O worker publica esses eventos no RabbitMQ, que entrega uma cópia para a fila exclusiva de cada réplica da API. Cada réplica, por sua vez, avisa os celulares conectados a ela por WebSocket.

Figura 1 – Arquitetura distribuída do ViaAgora



Fonte: elaborado pelos autores.

5.2 Módulos do servidor

Tabela 7 – Módulos do servidor

Arquivo	Responsabilidade
crossings.js	Catálogo das três travessias; varredura do site, extração do endereço do serviço, validação e normalização dos dados oficiais; simulação.
collect.js	Uma coleta completa: redescobre o endereço quando necessário, grava a leitura ou registra a falha.
collector.js	Processo do coletor: repete a coleta a cada minuto, com backoff exponencial e limpeza de leituras com mais de 30 dias.
domain.js	Regras de negócio: relatos, encerramento, gravação das leituras com eventos, estado da fonte, travessias com mediana e histórico.
app.js	Rotas HTTP, validação de consultas, autenticação por token, tratamento de erros e verificação de saúde.
broker.js	Assinatura da exchange fanout, envio por WebSocket, reconexão ao broker e publicação da outbox.
worker.js	Processo que publica a outbox a cada 750 ms.

Arquivo	Responsabilidade
db.js / migrate.js	Conexão ao PostgreSQL, migração com bloqueio consultivo e conversão das linhas para JSON.
seed-history.js	Geração de uma semana de histórico simulado para demonstração.

Fonte: elaborado pelos autores.

5.3 Modelo de dados

Tabela 8 – Tabelas do banco de dados

Tabela	Conteúdo
participants	Participação anônima de cada instalação: identificador e hash SHA-256 do token.
crossing_snapshots	Cada leitura de cada travessia: esperas A e B, embarcações, operação, tempo, período, aviso, origem (oficial ou simulado) e horários.
collector_state	Linha única com o modo do coletor, o endereço do serviço, a última tentativa, o último sucesso, o último erro e as falhas seguidas.
reports	Relatos: autor, travessia, lado, tipo, espera, descrição, coordenadas, situação, versão e chave de idempotência (única por autor).
outbox_events	Eventos a publicar: conteúdo JSON, tentativas, criação e publicação.

Fonte: elaborado pelos autores.

5.4 Interface de programação (API)

Tabela 9 – Rotas da API

Rota	Descrição
POST /v1/participants	Cria a participação anônima do aparelho e devolve o token uma única vez.
GET /v1/crossings	Travessias com terminais, última leitura, mediana da comunidade por lado, relatos ativos e estado da fonte.
GET /v1/crossings/:id/history	Média por hora do dia, série das últimas 6 horas e melhor hora de cada lado.
GET /v1/reports	Relatos recentes, com filtros por travessia, tipo, situação e período.
POST /v1/reports	Publica um relato; exige token e chave de idempotência.
PATCH /v1/reports/:id/resolve	Encerra um relato; somente o autor.
GET /health	Estado do banco, do broker e do coletor.
WS /v1/events	Canal de notificações: ready, resync, broker.status, crossing.updated, report.created, report.resolved e source.status.

Fonte: elaborado pelos autores.

5.5 Módulos do aplicativo Android

Tabela 10 – Módulos do aplicativo

Arquivo	Responsabilidade
MainActivity.kt	Telas de travessias, mapa e relatos; painéis de detalhes, relato, aviso e melhor horário; ciclo de vida.

Arquivo	Responsabilidade
Models.kt	Modelos de dados, catálogo das travessias, validação, faixas de espera, regra de disparo dos avisos e simulação.
RemoteRepository.kt	Comunicação HTTP e WebSocket com o servidor, cache local, idempotência dos envios e reconexão.
LocalRepository.kt	Modo demonstração: esperas simuladas, relatos e histórico guardados no aparelho.
JsonCodec.kt	Conversão entre JSON e os modelos.
MapPanel.kt	Mapa MapLibre com a camada de terminais e relatos.
HistoryChart.kt	Gráfico de barras da espera média por hora, desenhado diretamente no Canvas.
Alerts.kt	Armazenamento dos avisos e envio de notificações.
AppState.kt / Ui.kt	Modo e servidor ativos, descartando respostas antigas; componentes visuais e paleta.

Fonte: elaborado pelos autores.

5.6 Tratamento de erros

O tratamento de erros foi projetado para que o usuário sempre saiba o que aconteceu e o que pode fazer, sem mensagens técnicas. A API devolve códigos HTTP padronizados e uma mensagem em português, e nunca expõe detalhes internos em erros de servidor. A Tabela 11 resume as situações tratadas.

Tabela 11 – Tratamento de erros

Situação	Resposta do sistema
Dados inválidos no relato	HTTP 400 com "Confira travessia, lado, tipo, espera e coordenadas."; o aplicativo valida antes e marca o campo com erro.
Token ausente ou inválido	HTTP 401; o aplicativo descarta o token e cria nova participação automaticamente.
Encerrar relato de outra pessoa	HTTP 403; o aplicativo nem exibe o botão para quem não é o autor.
Relato ou travessia inexistente	HTTP 404 com mensagem descritiva.
Mesma chave de envio com outro conteúdo	HTTP 409, evitando sobrescrever um relato por engano.
Excesso de requisições	HTTP 429 pelo limite de taxa.
Banco indisponível	HTTP 503 com "Serviço temporariamente indisponível"; o /health indica a causa.
Broker indisponível	Gravações continuam; eventos ficam pendentes; o aplicativo mostra "Sincronização indisponível".
Fonte oficial fora do ar	Último dado mantido e marcado como atrasado; nova tentativa com intervalo crescente.
Sem conexão no celular	Dados da última consulta exibidos do cache; mensagem para tocar e tentar de novo.

Situação	Resposta do sistema
Mapa sem internet	Aviso "Mapa indisponível · use a lista"; as abas de travessias e relatos continuam funcionando.

Fonte: elaborado pelos autores.

5.7 Fluxos principais

Coleta de dados oficiais. A cada minuto o coletor consulta o serviço do painel. Na primeira vez, ou quando a leitura falha, ele antes varre a página da SEMIL para descobrir o endereço do serviço. Os dados são validados e filtrados para as três travessias. Em uma transação, cada leitura é gravada e, se algo visível mudou, um evento crossing.updated é colocado na outbox. O worker publica o evento, cada réplica o recebe pela sua fila e avisa os celulares, que então buscam as travessias atualizadas.

Publicação de um relato. O aplicativo gera uma chave de idempotência e envia o relato à réplica em que está conectado. A API valida os dados, grava o relato e o evento report.created na mesma transação e responde ao aplicativo. A partir daí, o caminho é o mesmo da coleta: worker, broker, todas as réplicas, todos os celulares. Se a resposta se perder e o aplicativo reenviar, a mesma chave faz a API devolver o relato já existente, sem duplicá-lo.

Recuperação de falhas. Quando a API perde a conexão com o broker, ela avisa os clientes (broker.status) e tenta reconectar a cada 1,5 s. Ao conseguir, envia resync, e os celulares buscam o estado completo, recuperando qualquer notificação perdida. Quando o celular perde o WebSocket, ele tenta de novo a cada 2,5 s e também consulta o estado ao voltar.

6 RELATÓRIO COM AS LINHAS DE CÓDIGO DO PROGRAMA

Esta seção apresenta as partes mais importantes do código-fonte, reproduzidas exatamente como estão nos arquivos do projeto, com a numeração de linhas original. O código completo acompanha a entrega. A Tabela 12 resume o tamanho de cada arquivo.

Tabela 12 – Linhas de código por arquivo

Arquivo	Parte	Linhas
server/src/crossings.js	Servidor	104
server/src/domain.js	Servidor	119
server/src/broker.js	Servidor	72
server/src/app.js	Servidor	65
server/src/collect.js	Servidor	27
server/src/collector.js	Servidor	18
server/src/seed-history.js	Servidor	23
server/src/db.js	Servidor	15
server/src/worker.js	Servidor	11
server/src/index.js	Servidor	11
server/src/migrate.js	Servidor	3
server/migrations/001.sql	Banco de dados	61
android/…/MainActivity.kt	Android	430
android/…/Models.kt	Android	103
android/…/RemoteRepository.kt	Android	102
android/…/MapPanel.kt	Android	102
android/…/LocalRepository.kt	Android	57
android/…/Ui.kt	Android	119
android/…/JsonCodec.kt	Android	52
android/…/HistoryChart.kt	Android	40
android/…/Alerts.kt	Android	37
android/…/AppState.kt	Android	26
server/test/api.test.js	Testes	98
server/test/collector.test.js	Testes	77
server/test/events.test.js	Testes	95
android/…/test/DomainTest.kt	Testes	78
Total		1945

Fonte: elaborado pelos autores.

6.1 Varredura do site oficial

As funções abaixo reproduzem o caminho do navegador. findServiceScript localiza na página o script do painel; findFeedBase lê o script linha a linha, ignora as comentadas e extrai o endereço ativo do serviço; discoverFeedUrl encadeia as duas etapas.

server/src/crossings.js (linhas 28 a 53)

```
28  export function findServiceScript(html,base=SITE_URL){
29    const src=html.match(/<script[^>]+src="([^"]*travessia-service[^"]*)"/i)?.[1];
30    return src?new URL(src,base).toString():null;
31  }
32  export function findFeedBase(script){
33    for(const line of script.split(/\r?\n/)){
34      if(line.trim().startsWith('//'))continue;
35      const url=line.match(/getInfo\s*=\s*["'](https?:\/\/[^"']+)["']/)?.[1];
36      if(url)return url.endsWith('/')?url:url+'/';
37    }
38    return null;
39  }
40
41  async function download(fetchImpl,url,type){
42    const response=await fetchImpl(url,{headers,signal:AbortSignal.timeout(15000)});
43    if(!response.ok)throw new Error(`Fonte respondeu ${response.status} em ${new URL(url).ho
    st}`);
44    return type==='json'?response.json():response.text();
45  }
46  export async function discoverFeedUrl(fetchImpl=fetch){
47    const html=await download(fetchImpl,SITE_URL,'text');
48    const script=findServiceScript(html);
49    if(!script)throw new Error('Script do painel não encontrado na página.');
50    const base=findFeedBase(await download(fetchImpl,script,'text'));
51    if(!base)throw new Error('Endereço do serviço não encontrado no script.');
52    return base+'PT';
53  }
```

A função parseFeed valida cada item com o esquema Zod, descarta os inválidos, mantém só as travessias do catálogo e normaliza os campos. O aviso fixo do 0800 é descartado, porque aparece em todas as travessias e não descreve a situação atual.

server/src/crossings.js (linhas 72 a 90)

```
72  const notice=value=>{const text=stripHtml(value).slice(0,400);return text&&!genericNotice.
    test(text)?text:null;};
73  export function parseFeed(data){
74    if(!Array.isArray(data))throw new Error(typeof data==='string'?`Fonte recusou a consulta
    : ${data}`:'Formato inesperado da fonte.');
75    const result=[];
76    for(const raw of data){
77      const item=feedItem.safeParse(raw);
78      if(!item.success)continue;
79      const d=item.data,id=slug(`${d.TravessiaPontoA} ${d.TravessiaPontoB}`);
80      if(!crossingById(id))continue;
81      result.push({crossingId:id,waitA:d.EsperaPontoA,waitB:d.EsperaPontoB,boats:d.NumEmbarc
    a,operating:d.EmbarcaAtivo&&d.NumEmbarca>0,
82        weatherA:stripHtml(d.ClimaA)||null,weatherB:stripHtml(d.ClimaB)||null,period:stripHt
    ml(d.DiaNoite)||null,
83        notice:notice(d.AvisoUsuarioApp||d.AvisoUsuario),sourceTime:parseSourceTime(d.DataCo
    nsulta)});
84    }
85    return result;
86  }
87
88  export async function fetchOfficial(fetchImpl=fetch,feedUrl=DEFAULT_FEED_URL){
89    return parseFeed(await download(fetchImpl,feedUrl,'json'));
```

```
90  }
```

6.2 Coleta com tolerância a falhas

A coleta tenta o endereço conhecido e, se falhar, varre o site de novo. Se a varredura também falhar, usa o endereço de reserva. Qualquer falha é registrada no banco sem apagar a última leitura válida.

server/src/collect.js (linhas 1 a 27)

```
1   import { discoverFeedUrl,fetchOfficial,simulate,DEFAULT_FEED_URL } from './crossings.js';
2   import { recordSnapshots,recordFailure } from './domain.js';
3
4   // Uma coleta: descobre o endereço do serviço varrendo o site (só quando ainda não sabe ou
     quando a leitura falha),
5   // lê os dados, normaliza e grava. Falhas ficam registradas e o último dado bom continua d
    isponível na API.
6   export function createCollector({pool,mode='oficial',fetchImpl=fetch,feedUrl=null,log=cons
    ole}){
7     let url=feedUrl;
8     async function resolveUrl(force){
9       if(url&&!force)return url;
10      try{url=await discoverFeedUrl(fetchImpl);log.info?.('Serviço das travessias encontrado
     no site:',url);}
11      catch(error){log.warn?.('Varredura do site falhou, usando endereço conhecido:',error.m
    essage);url=url||DEFAULT_FEED_URL;}
12      return url;
13    }
14    return {
15      get feedUrl(){return url;},
16      async run(now=new Date()){
17        if(mode==='simulado')return recordSnapshots(pool,simulate(now),'simulado',null);
18        try{
19          let statuses;
20          try{statuses=await fetchOfficial(fetchImpl,await resolveUrl(false));}
21          catch{statuses=await fetchOfficial(fetchImpl,await resolveUrl(true));}
22          if(!statuses.length)throw new Error('Nenhuma travessia da Baixada Santista na resp
    osta da fonte.');
23          return await recordSnapshots(pool,statuses,'oficial',url);
24        }catch(error){await recordFailure(pool,mode,error.message);throw error;}
25      },
26    };
27  }
```

O processo do coletor repete a coleta e, a cada falha seguida, dobra o intervalo de espera, até o máximo de 10 minutos, para não sobrecarregar a fonte oficial.

server/src/collector.js (linhas 1 a 18)

```
1   import {createPool,migrate} from './db.js';
2   import {createCollector} from './collect.js';
3   const pool=createPool();await migrate(pool);let stopping=false;
4   process.on('SIGINT',()=>{stopping=true;});process.on('SIGTERM',()=>{stopping=true;});
5   const mode=process.env.COLLECTOR_MODE==='simulado'?'simulado':'oficial';
6   const interval=Math.max(30_000,Number(process.env.COLLECT_INTERVAL_MS||60_000));
7   const collector=createCollector({pool,mode,feedUrl:process.env.TRAVESSIAS_FEED_URL||null})
    ;
8   console.log(`ViaAgora: coletor de travessias iniciado (modo ${mode}, a cada ${interval/100
    0}s).`);
9   let failures=0,runs=0;
10  while(!stopping){
11    try{const {saved,changes}=await collector.run();failures=0;console.log(`Coleta: ${saved}
     travessias, ${changes} com mudança.`);}
12    catch(error){failures++;console.error('Coleta falhou; último dado preservado:',error.mes
    sage);}
13    if(++runs%60===0)await pool.query("DELETE FROM crossing_snapshots WHERE collected_at<now
    ()-interval '30 days'").catch(()=>{});
14    // Em falha, espera mais a cada tentativa (até 10 min) para não sobrecarregar a fonte of
    icial.
```

```
15    const delay=Math.min(interval*2**Math.min(failures,4),600_000);
16    for(let waited=0;waited<delay&&!stopping;waited+=500)await new Promise(resolve=>setTimeo
    ut(resolve,500));
17  }
18  await pool.end();
```

6.3 Gravação das leituras com outbox transacional

Cada leitura é gravada na mesma transação do seu evento. O evento só é criado quando algo visível mudou, para não acordar os aparelhos sem necessidade. A primeira falha e a recuperação da fonte também geram eventos, para que o aplicativo informe o usuário.

server/src/domain.js (linhas 58 a 84)

```
58  const changed=(previous,next)=>!previous||['wait_a','wait_b','boats','operating','weather_
    a','weather b','notice'].some(field=>previous[field]!==next[field]);
59  // Grava uma leitura de cada travessia. O evento só sai quando algo visível mudou, para nã
    o acordar os celulares à toa.
60  export async function recordSnapshots(pool,statuses,source,feedUrl=null){
61    return transaction(pool,async client=>{
62      let changes=0;
63      for(const s of statuses){
64        const row={wait_a:s.waitA,wait_b:s.waitB,boats:s.boats,operating:s.operating,weather
    _a:s.weatherA,weather_b:s.weatherB,notice:s.notice};
65        const previous=(await client.query('SELECT * FROM crossing_snapshots WHERE crossing_
    id=$1 ORDER BY collected_at DESC LIMIT 1',[s.crossingId])).rows[0];
66        await client.query(`INSERT INTO crossing_snapshots(crossing_id,wait_a,wait_b,boats,o
    perating,weather_a,weather_b,period,notice,source,source_time)
67          VALUES($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11)`,[s.crossingId,s.waitA,s.waitB,s.boats,
    s.operating,s.weatherA,s.weatherB,s.period,s.notice,source,s.sourceTime]);
68        if(changed(previous,row)){changes++;await appendEvent(client,{type:'crossing.updated
    ',crossingId:s.crossingId});}
69      }
70      const state=(await client.query('SELECT failures,last_success_at FROM collector_state
    WHERE id=1 FOR UPDATE')).rows[0];
71      await client.query(`INSERT INTO collector_state(id,mode,feed_url,last_attempt_at,last_
    success_at,last_error,failures) VALUES(1,$1,$2,now(),now(),NULL,0)
72        ON CONFLICT(id) DO UPDATE SET mode=$1,feed_url=COALESCE($2,collector_state.feed_url)
    ,last_attempt_at=now(),last_success_at=now(),last_error=NULL,failures=0`,[source,feedUrl])
    ;
73      if(state?.failures>0)await appendEvent(client,{type:'source.status',available:true});
74      return {saved:statuses.length,changes};
75    });
76  }
77  export async function recordFailure(pool,mode,message){
78    return transaction(pool,async client=>{
79      const state=(await client.query(`INSERT INTO collector_state(id,mode,last_attempt_at,l
    ast_error,failures) VALUES(1,$1,now(),$2,1)
80        ON CONFLICT(id) DO UPDATE SET mode=$1,last_attempt_at=now(),last_error=$2,failures=c
    ollector_state.failures+1 RETURNING failures`,[mode,String(message).slice(0,300)])).rows[0
    ];
81      if(state.failures===1)await appendEvent(client,{type:'source.status',available:false})
    ;
82      return state.failures;
83    });
84  }
```

6.4 Publicação dos eventos e entrega às réplicas

O worker usa FOR UPDATE SKIP LOCKED para reservar os eventos pendentes e só os marca como publicados depois da confirmação do RabbitMQ. Se algo falhar, a transação é desfeita e os eventos continuam pendentes.

server/src/broker.js (linhas 56 a 72)

```
56  export async function publishBatch(pool,url=rabbitUrl()){
57    const client=await pool.connect();let connection,channel;
58    try{
59      await client.query('BEGIN');
60      const pending=await client.query('SELECT id,payload FROM outbox_events WHERE published
    _at IS NULL ORDER BY created_at LIMIT 25 FOR UPDATE SKIP LOCKED');
61      if(!pending.rowCount){await client.query('COMMIT');return 0;}
62      connection=await amqp.connect(url,{timeout:2000});connection.on('error',()=>{});
63      channel=await connection.createConfirmChannel();channel.on('error',()=>{});
64      await channel.assertExchange(exchange,'fanout',{durable:true});
65      for(const row of pending.rows){
66        await new Promise((resolve,reject)=>channel.publish(exchange,'',Buffer.from(JSON.str
    ingify(row.payload)),{persistent:true,messageId:row.id,contentType:'application/json'},err
    or=>error?reject(error):resolve()));
67        await client.query('UPDATE outbox_events SET published_at=now(),attempts=attempts+1
    WHERE id=$1',[row.id]);
68      }
69      await client.query('COMMIT');return pending.rowCount;
70    }catch(error){await client.query('ROLLBACK');throw error;}
71    finally{await safelyClose(channel);await safelyClose(connection);client.release();}
72  }
```

Cada réplica cria uma fila exclusiva ligada à exchange fanout e repassa cada mensagem aos seus clientes WebSocket. Ao conectar ou reconectar, envia resync para que os clientes busquem o estado completo.

server/src/broker.js (linhas 29 a 46)

```
29    async function connect(){
30      try{
31        connection=await amqp.connect(url,{timeout:3000});
32        connection.on('error',()=>{});connection.on('close',retry);
33        if(stopping){await safelyClose(connection);return;}
34        channel=await connection.createChannel();channel.on('error',()=>{});
35        await channel.assertExchange(exchange,'fanout',{durable:true});
36        const queue=await channel.assertQueue('',{exclusive:true,autoDelete:true});
37        await channel.bindQueue(queue.queue,exchange,'');
38        await channel.consume(queue.queue,message=>{
39          if(!message)return;
40          try{broadcast(JSON.parse(message.content.toString()));channel.ack(message);}
41          catch{channel.nack(message,false,false);}
42        });
43        connected=true;
44        broadcast({type:'resync',connected:true});
45      }catch{await safelyClose(connection);retry();}
46    }
```

6.5 Comunicação no aplicativo Android

No aplicativo, a busca de dados roda em uma thread de trabalho e devolve o resultado à thread principal. O WebSocket trata os tipos de evento e reconecta automaticamente após 2,5 s em caso de falha.

android/app/src/main/java/br/com/viaagora/RemoteRepository.kt (linhas 56 a 62)

```
56      fun fetch(onSuccess: (Snapshot) -> Unit, onError: (String) -
    > Unit) = submit(onError) {
57          val crossingsText = request("GET", "/v1/crossings").toString()
58          val reports = decodeReports(request("GET", "/v1/reports?limit=500").getJSONArray("
    reports").toString())
59          val (crossings, source) = decodeCrossings(crossingsText)
60          prefs.edit().putString("crossings:$base", crossingsText).putString("reports:$base"
    , encodeReports(reports)).apply()
61          main.post { if (!closed) onSuccess(Snapshot(crossings, reports, source)) }
62      }
```

android/app/src/main/java/br/com/viaagora/RemoteRepository.kt (linhas 82 a 100)

```
82      fun connect(onChange: () -> Unit, onConnection: (Boolean) -> Unit) {
83          if (closed) return
84          val address = base.replaceFirst("https://", "wss://").replaceFirst("http://", "ws
    ://") + "/v1/events"
85          socket = client.newWebSocket(Request.Builder().url(address).build(), object: WebS
    ocketListener() {
86              override fun onMessage(webSocket: WebSocket, text: String) {
87                  val event = runCatching { JSONObject(text) }.getOrNull() ?: return
88                  main.post { if (!closed) {
89                      when (event.optString("type")) {
90                          "ready", "resync" -
    > { onConnection(event.optBoolean("connected", true)); onChange() }
91                          "broker.status" -
    > onConnection(event.optBoolean("connected", false))
92                          else -> onChange()
93                      }
94                  } }
95              }
96              override fun onFailure(webSocket: WebSocket, t: Throwable, response: Response
    ?) { reconnect() }
97              override fun onClosed(webSocket: WebSocket, code: Int, reason: String) { reco
    nnect() }
98              private fun reconnect() { main.post { if (!closed) { onConnection(false); mai
    n.postDelayed({ connect(onChange, onConnection) }, 2500) } } }
99          })
100     }
```

A regra dos avisos só considera leituras atuais de travessias em operação. Ela é uma função pura, testada sem o Android, e chamada a cada atualização recebida.

android/app/src/main/java/br/com/viaagora/Models.kt (linhas 79 a 86)

```
79  // Alerta "me avise quando a espera ficar abaixo de N minutos": dispara uma vez e é removi
    do.
80  data class WaitAlert(val crossingId: String, val side: String, val maxMinutes: Int)
81  fun triggeredAlerts(alerts: List<WaitAlert>, crossings: List<Crossing>): List<Pair<WaitAle
    rt, Int>> = alerts.mapNotNull { alert ->
82      val status = crossings.firstOrNull { it.id == alert.crossingId }?.status ?: return@map
    NotNull null
83      if (status.stale || !status.operating) return@mapNotNull null
84      val wait = if (alert.side == "A") status.waitA else status.waitB
85      if (wait <= alert.maxMinutes) alert to wait else null
86  }
```

android/app/src/main/java/br/com/viaagora/MainActivity.kt (linhas 118 a 126)

```
118     private fun update(snapshot: Snapshot) {
119         data = snapshot
120         for ((alert, wait) in triggeredAlerts(alerts.list(), snapshot.crossings)) {
121             val crossing = snapshot.crossings.first { it.id == alert.crossingId }
122             val text = "Espera em ${crossing.terminal(alert.side).name}: $wait min (seu l
    imite: ${alert.maxMinutes} min)."
123             notifier.show(alert.hashCode(), crossing.name, text); toast("${crossing.name}
     · $text"); alerts.remove(alert)
124         }
125         render()
126     }
```

7 APRESENTAÇÃO DO PROGRAMA EM FUNCIONAMENTO

Esta seção é o roteiro da apresentação do programa em um computador, mostrando todas as funcionalidades pedidas e as extras. A demonstração dura cerca de 15 minutos e foi pensada para três integrantes, um em cada bloco. As figuras foram capturadas do aplicativo em execução no emulador Android 13, ligado ao servidor real.

7.1 Preparação (antes da apresentação)

1. Iniciar o Docker Desktop e, na pasta do projeto, executar docker compose up --build -d. Conferir com docker compose ps que os seis serviços estão ativos.
2. Gerar o histórico de demonstração com docker compose exec api node src/seed-history.js 7.
3. Conferir em docker compose logs collector a mensagem "Coleta: 3 travessias". Se o site oficial estiver fora do ar, usar COLLECTOR_MODE=simulado.
4. Abrir o emulador (ou ligar o celular na mesma rede Wi-Fi), instalar o APK e deixar abertos o site da SEMIL e o painel do RabbitMQ (http://localhost:15689).

7.2 Roteiro

Tabela 13 – Roteiro da apresentação

Tempo	Responsável	O que mostrar	O que explicar
0–2 min	Aluno 1	Site da SEMIL com o painel das travessias.	O problema: filas, trânsito e emissões; o painel não guarda histórico nem avisa.
2–4 min	Aluno 1	Diagrama da arquitetura (Figura 1).	Os seis processos, o papel do banco como fonte de verdade e do broker.
4–6 min	Aluno 2	App em modo conectado: cartões das travessias (Figura 2).	Dados oficiais coletados pelo scraper, cor da espera, embarcações, aviso de maré e idade do dado.
6–8 min	Aluno 2	Melhor horário e mapa (Figuras 3 e 4).	Histórico por hora e terminais coloridos pela espera.
8–10 min	Aluno 2	Publicar relato de fila com espera de 45 min (Figura 5).	A estimativa da comunidade aparece ao lado da oficial; idempotência do envio.
10–12 min	Aluno 3	Segundo aparelho na outra réplica (porta 4311) recebendo o relato.	Exchange fanout com uma fila por réplica, mostrada no painel do RabbitMQ.

Tempo	Responsável	O que mostrar	O que explicar
12–14 min	Aluno 3	Falhas: parar o broker, publicar, religar; parar uma réplica.	Outbox transacional, reconexão e ressincronização; nenhum dado é perdido.
14–15 min	Aluno 3	Aviso de espera e modo demonstração.	Funções extras e encerramento.

Fonte: elaborado pelos autores.

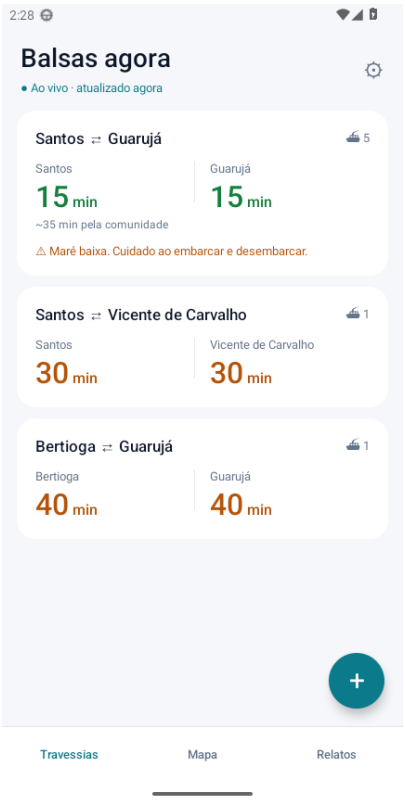


Figura 2 – Travessias no modo conectado



Figura 3 – Melhor horário

Fonte: elaborado pelos autores (captura do aplicativo).

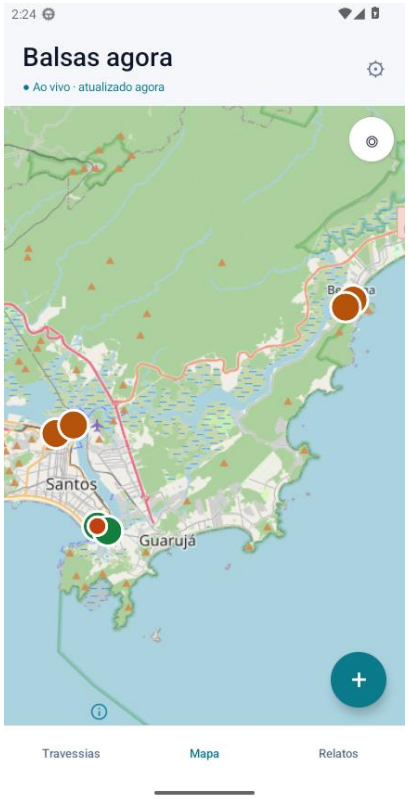




Figura 4 – Mapa dos terminais *Figura 5 – Formulário de relato*

Fonte: elaborado pelos autores (captura do aplicativo).

7.3 Demonstração das falhas

A parte mais importante para a disciplina é mostrar que o sistema continua coerente quando um componente falha. Recomenda-se executar, nesta ordem:

- **Queda do broker:** docker compose stop broker. Publicar um relato: ele é salvo e o aplicativo indica que a sincronização está indisponível. Com docker compose start broker, o worker publica o evento pendente e os aparelhos se atualizam sozinhos.
- **Queda de uma réplica:** docker compose stop api-2. O aparelho ligado à réplica 1 continua funcionando normalmente. Ao subir a réplica 2, o outro aparelho reconecta e recupera o estado.
- **Queda da fonte oficial:** docker compose stop collector. A API continua servindo o último dado e, depois de 10 minutos, o aplicativo o marca como atrasado. Com docker compose start collector, os dados voltam a ser atualizados.

Não se deve apagar os volumes do Docker durante a apresentação, para que fique claro que os dados sobrevivem aos reinícios.

8 CONSIDERAÇÕES FINAIS

O ViaAgora atingiu o objetivo proposto: um aplicativo Android que informa, em tempo real, a espera das travessias da Baixada Santista, apoiado em um servidor distribuído que combina o dado oficial com os relatos da comunidade. A arquitetura mostrou, na prática, os conceitos estudados na disciplina. O banco de dados como fonte de verdade, a outbox transacional, a exchange fanout, a idempotência e a ressincronização dos clientes permitiram derrubar o broker, uma réplica da API e a fonte oficial sem perda de dados. Os 20 testes de integração do servidor e os 8 testes do aplicativo comprovam esses comportamentos de forma repetível.

O projeto tem limitações conhecidas. O serviço do painel oficial não é uma API aberta e documentada, por isso pode mudar ou bloquear as consultas. O coletor foi construído para detectar essa situação e o modo simulado garante a demonstração. As coordenadas dos terminais são aproximadas, os avisos dependem de o aplicativo estar aberto ou em segundo plano recente, e não há moderação dos relatos. Como trabalhos futuros, propõem-se a hospedagem do servidor com HTTPS, o envio de notificações por um serviço de push, a previsão da espera a partir do histórico e da maré, e a reputação dos usuários para ponderar os relatos.

Quanto ao efeito na formação do grupo, o trabalho aproximou a teoria de sistemas distribuídos de um problema real da região onde os integrantes vivem. Foi preciso lidar com uma fonte de dados que não estava sob nosso controle, com falhas parciais e com a diferença entre "funcionar no computador" e "funcionar quando algo dá errado". O uso de testes que reproduzem falhas reais, como derrubar a conexão com o broker no meio da execução, mudou a forma de pensar a qualidade de um sistema. A integração entre aplicativo móvel, rede, banco de dados, mensageria e contêineres exercitou de forma conjunta conteúdos que costumam ser estudados separadamente ao longo do curso.

REFERÊNCIAS

ANDROID DEVELOPERS. **Guia do desenvolvedor Android**. Mountain View: Google, 2026. Disponível em: https://developer.android.com/guide. Acesso em: 28 out. 2026.

COULOURIS, George; DOLLIMORE, Jean; KINDBERG, Tim; BLAIR, Gordon. **Sistemas distribuídos**: conceitos e projeto. 5. ed. Porto Alegre: Bookman, 2013.

DOCKER INC. **Docker Compose overview**. 2026. Disponível em: https://docs.docker.com/compose/. Acesso em: 28 out. 2026.

FASTIFY. **Fastify documentation**. 2026. Disponível em: https://fastify.dev/docs/latest/. Acesso em: 28 out. 2026.

FETTE, Ian; MELNIKOV, Alexey. **RFC 6455**: The WebSocket Protocol. Internet Engineering Task Force, 2011. Disponível em: https://www.rfc-editor.org/rfc/rfc6455. Acesso em: 28 out. 2026.

FIELDING, Roy Thomas. **Architectural styles and the design of network-based software architectures**. 2000. Tese (Doutorado em Information and Computer Science) – University of California, Irvine, 2000.

HOHPE, Gregor; WOOLF, Bobby. **Enterprise integration patterns**: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003.

JETBRAINS. **Kotlin documentation**. 2026. Disponível em: https://kotlinlang.org/docs/home.html. Acesso em: 28 out. 2026.

KLEPPMANN, Martin. **Designing data-intensive applications**. Sebastopol: O'Reilly, 2017.

MAPLIBRE. **MapLibre Native for Android**. 2026. Disponível em: https://maplibre.org/maplibre-native/android/examples/. Acesso em: 28 out. 2026.

OPENSTREETMAP FOUNDATION. **Copyright and license**. 2026. Disponível em: https://www.openstreetmap.org/copyright. Acesso em: 28 out. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL 16 documentation**. 2026. Disponível em: https://www.postgresql.org/docs/16/. Acesso em: 28 out. 2026.

RABBITMQ. **RabbitMQ tutorials and documentation**: publish/subscribe and publisher confirms. 2026. Disponível em: https://www.rabbitmq.com/docs. Acesso em: 28 out. 2026.

RICHARDSON, Chris. **Microservices patterns**: with examples in Java. Shelter Island: Manning, 2018.

SÃO PAULO (Estado). Secretaria de Meio Ambiente, Infraestrutura e Logística. **Travessias Litorâneas**. São Paulo: SEMIL, 2026. Disponível em: https://semil.sp.gov.br/travessias/. Acesso em: 28 out. 2026.

TANENBAUM, Andrew S.; VAN STEEN, Maarten. **Sistemas distribuídos**: princípios e paradigmas. 2. ed. São Paulo: Pearson Prentice Hall, 2007.